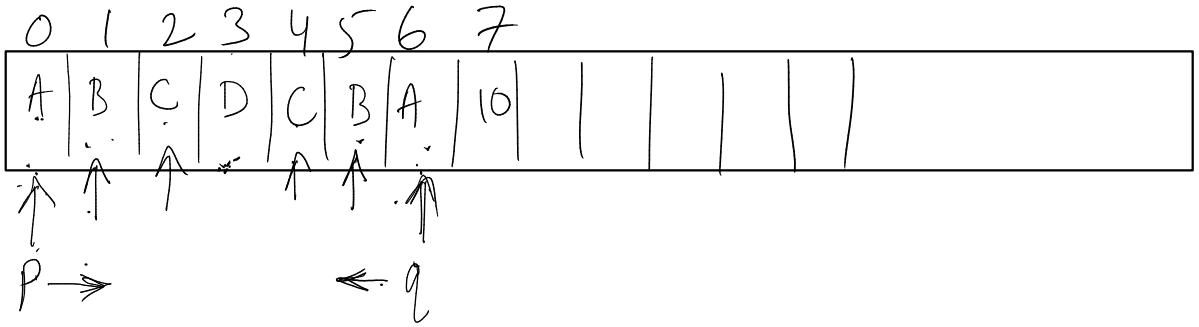
W.A.P to test the given string is palindrome or not.

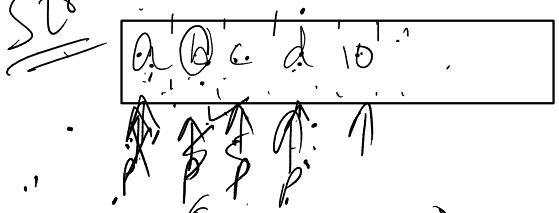
→ Without extra Array

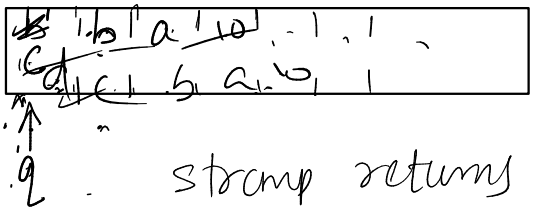


P < q

```
p = str; q = str + strlen(str) - 1;
while(p<q) {  if( *p != *q)
                  break;

p++; q--;
}
if(p<q)  pf("Not palindrome");
else     pf("---");
```



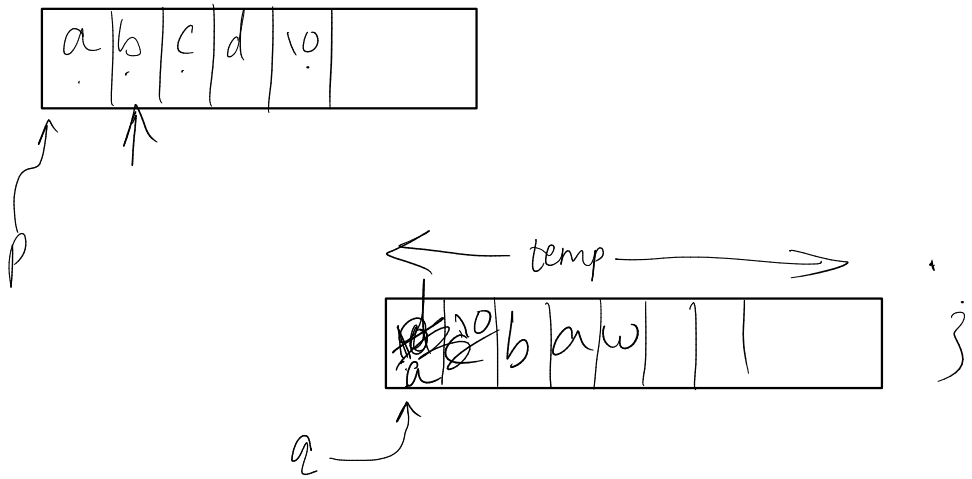


strRev(str, temp); ✓

strcmp returns
0 ⇒ when exactly same
≤0 or >0 ⇒ when not same

ret = strcmp(str, temp);
if(ret == 0) pf("palindrome"); else pf("NOT");

```
void strRev (const char *p, char *q)
{
    *q = '\0';
    while(*p) {
        mm(q+1, q, strlen(q)+1);
        *q = *p;
        p++;
    }
}
```

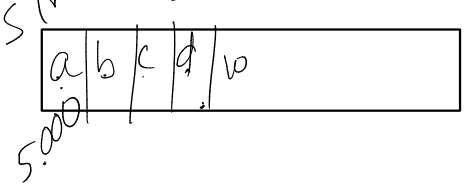


s1 "good"

s2 "bad"

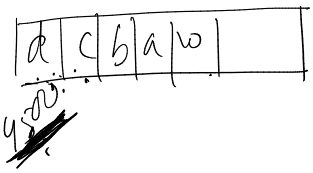
n = strcmp(s1, s2);

```
main() { char str[30]; puts("—"); gets(str);
if( strcmp( str, strRev(str)) ==0)
    Pf("palindrome");
else
    Pf("Not Palindrome");
}
```




```
char *strRev (const char *p)
{
    static char temp[30];
    return temp;
}
```





char *strstr(const char *, const char *);
s1 s2

searches the occurrence of s2 in s1.

If not found, returns NULL

If found, returns the location of first occurrence

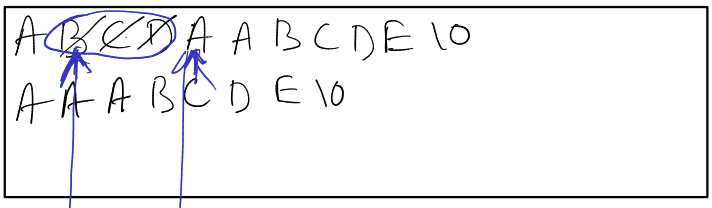
```
char *ptr;
gets(s1); gets(s2);
ptr = strstr(s1, s2);
if(ptr == NULL) pf("---");
else pf("found at %u", ptr - s1);
```

s1 => "ABCDABCDE"
500
s2 => "BCD"

W.A.P to Delete the 1st occurrence of substring s2, in a larger string s1.

```
gets(s1);
gets(s2);
ptr = strstr(s1, s2);
if(ptr){ src = ptr + strlen(s2);
memmove(ptr, ptr+len(s2), strlen(ptr+strlen(s2))+1);
                          strlen(src)+1
```

s1



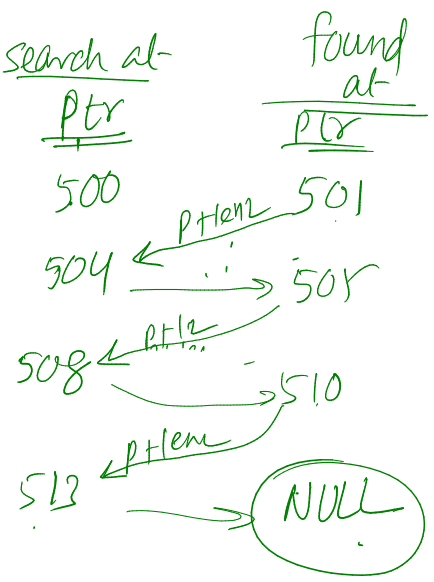
s2 => "BCD"

W.A.P to Delete the last occurrence of s2 in s1

s1 => "abcdabcdeabcdef"
s2 => "bcd"

search at ptr: 500

found at ptr: 501




```
char *temp, *found = NULL;
temp = s1; l2 = strlen(s2);
while( temp = strstr(temp, s2) )
  {
    found = temp;
    temp += l2;
  }
if(found){
    mm(      );
    }
```

MUST TRY

Dyn mem Alloc
2D Arr
Adv Pointer

-> W.A.P to remove All occurrences of s2 in s1.

-> W.A.P to replace all occurrences of s2 with Reverse eg. of s2 in s1

s1 -> "abcdeabcde"
s2 -> "bcd"

use: strstr, strncpy, strRev
lib U.D.

-> W.A.P to Hide all occurrences of s2 in s1, with *s.

s1 -> "abcdeabcde" s2 -> "bcd"

-> use strstr, memset
lib